

★ REDUX TOOLKIT (RTK) - TOPPER NOTES ★

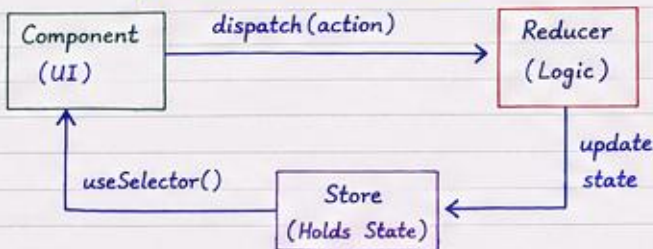
Page 1

1. What is Redux ?

Redux is a state management library for JavaScript apps.

It stores the entire application state in a single central store.

Redux Flow (Mental Model)



2. Core Building Block - SLICE

A slice is a collection of:

- initial state
- reducers
- generated action creators
- other reducer logic

★
`createSlice()`
from `@reduxjs/toolkit` is used
to create a slice.

3. Key Terms

- Store → central place that holds the state
- Slice → state + reducers + actions
- Reducer → function that decides how state changes based on action
- Action → object that describes what happened
- Dispatch → sends an action to the store
- Selector → reads data from the store
- Payload → data passed with an action
- Immer → RTK uses Immer internally to write "mutating" code safely.

4. createSlice() - Example (counterSlice.js)

```
import { createSlice } from "@reduxjs/toolkit"

const counterSlice = createSlice({
  name: 'counter',
  initialState: {
    value: 0,
    name: 'Kapil'
  },
  reducers: {
    increment: (state) => {
      state.value += 1
    },
    decrement: (state) => {
      if (state.value > 0) state.value -= 1
    },
    reset: (state) => {
      state.value = 0
    },
    changeName: (state, action) => {
      state.name = action.payload
    }
  }
})
```

We can write mutating code like `state.value++` because Immer handles immutability internally.

```
export const { increment, decrement, reset, changeName }
      = counterSlice.actions
export default counterSlice.reducer
```

5. Configure Store (store.js)

```
import { configureStore } from '@reduxjs/toolkit'
import counterReducer from '../features/counter/counterSlice'

const store = configureStore({
  reducer: {
    counter: counterReducer // key = "counter"
  }
})

export default store
```

State structure will be:

```
state.counter
  .value
state.counter
  .name
```

6. Provide Store to React App - Provider

```
import { Provider } from 'react-redux'
import store from './store'
import App from './App'
```

```
<Provider store={store}>
  <App />
</Provider>
```

Provider makes the store available to all components in the app.

7. Using Redux in Components

A. useSelector() - Read State

```
const value = useSelector(
  (state) => state.counter.value
)
```

useSelector always takes the latest state from the store.

B. useDispatch() - Send Action

```
const dispatch = useDispatch()
```

- dispatch(increment())
 - dispatch(decrement())
 - dispatch(reset())
 - dispatch(changeName("Rahul"))
- } Action Creators (from slice)

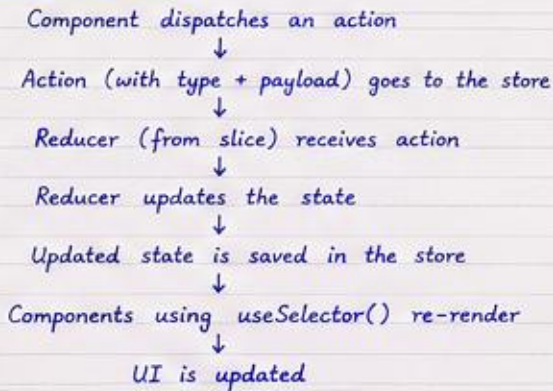
8. Action & Payload

When we pass some data while dispatching an action, it is available in reducer as action.payload

```
dispatch(changeName('Rahul')) → Reducer
                                state.name = action.payload
                                                ('Rahul')
```

★ Payload helps us send data with an action.

9. Complete Flow



★ Golden Rules

- ① READ → useSelector()
- ② WRITE → useDispatch()
- ③ ACTION → describes what happened
- ④ PAYLOAD → data with action
- ⑤ REDUCER → updates state
- ⑥ SLICE → state + reducers + actions
- ⑦ IMMER → safe mutating code in reducers

★ Understand the flow, not just the code. ★
That's the key to mastering Redux Toolkit! 😊

11. Quick Revision CheatSheet

- `createSlice()` → create slice (state + reducers + actions)
- `configureStore()` → create store and add reducers
- `Provider` → makes store available in app
- `useSelector()` → read state
- `useDispatch()` → send action
- `action.payload` → carries data
- `Immer` → write mutating code safely

Example Usage in Component

```
const value = useSelector(
  (state) => state.counter.value
)
const dispatch = useDispatch()

return (
  <div>
    <p>Count: {value}</p>
    <button onClick={() => dispatch(increment())}></button>
    <button onClick={() => dispatch(decrement())}></button>
    <button onClick={() => dispatch(reset())}>Reset</button>
  </div>
)
```

Remember

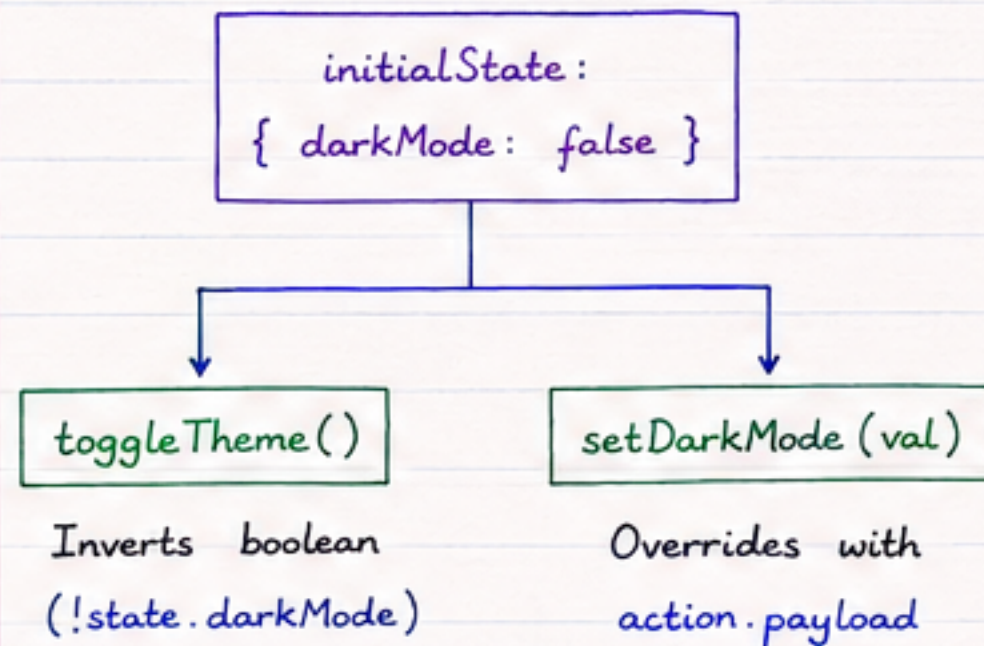
Redux Toolkit
reduces boilerplate
and makes Redux
simple, powerful
and enjoyable!



REDUX TOOLKIT (RTK) - THEME SLICE (themeSlice.js)

1. PURPOSE & OVERVIEW

- Role: Manages global UI theme state across the entire React app.
- Goal: Toggle between Dark Mode (true) and Light Mode (false), or explicitly assign a specific boolean state via payload.



MENTAL MODEL

Your UI doesn't store theme.
It asks Redux →
"What is the current theme?"
Redux answers →
"darkMode is true/false."
UI updates accordingly!



2. ANATOMY OF 'themeSlice.js'

createSlice() combines:

- ★ Slice Name: "theme"
(used as action prefix,
e.g., `theme/toggleTheme`)
- ★ Initial State: { darkMode: false }
- ★ Reducers: Case functions that update the state using Immer.js.

3. CODE IMPLEMENTATION

```
import { createSlice } from "@reduxjs/toolkit";

const initialState = {
  darkMode: false, // Default is Light Mode
};

const themeSlice = createSlice({
  name: "theme",
  initialState,
  reducers: {
    // Action 1: Toggle current boolean value
    toggleTheme: (state) => {
      state.darkMode = !state.darkMode;
    },
    // Action 2: Directly set to a specific boolean (true/false)
    setDarkMode: (state, action) => {
      state.darkMode = action.payload;
    },
  },
});

// Export Action Creators (for dispatching in UI)
export const { toggleTheme, setDarkMode } = themeSlice.actions;

// Export Reducer (for store registration)
export default themeSlice.reducer;
```

4. INTEGRATION CHECKLIST (Store → UI → DOM)

Step 1: Register in 'store.js'

```
reducer: {
  theme: themeReducer // State path: state.theme.darkMode
}
```

Step 2: Read & Dispatch in Component (ThemeToggle.jsx)

```
const darkMode = useSelector((state) => state.theme.darkMode);
const dispatch = useDispatch();
<button onClick={() => dispatch(toggleTheme())}> Toggle </button>
```

Step 3: Sync to HTML for Tailwind (App.jsx)

```
useEffect(() => {
  document.documentElement.classList.toggle("dark", darkMode);
}, [darkMode]);
```

5. COMMON PITFALLS & CHEAT SHEET

- ★ Redux Toolkit Uses Immer:
Direct mutations like `state.darkMode = ...` are completely safe inside `createSlice` reducers.
- ★ Action Type Strings:
 - `dispatch(toggleTheme())`
→ { type: 'theme/toggleTheme' }
 - `dispatch(setDarkMode(true))`
→ { type: 'theme/setDarkMode', payload: true }
- ★ Tailwind Class Strategy:
Don't forget `'darkMode: class'` (or `'selector'`) in your CSS configuration for the `'dark:'` utility variants to trigger.

QUICK RECAP

ACTIONS

```
toggleTheme()
→ Flip (true ↔ false)

setDarkMode(val)
→ Set to specific value (true/false)
```

STATE PATH

`state.theme.darkMode`

Single source of truth for theme in the app!



REDUX TOOLKIT (RTK) ASYNC - CHEAT SHEET ★

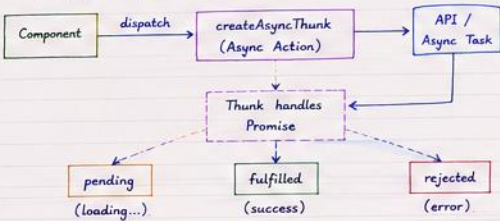
Page 1

1. Why Async in Redux?

- APIs, DB calls, timeouts → all are asynchronous.
- We need to handle → loading, success, error properly in the store.

2. The Hero - createAsyncThunk()

- Built-in function in RTK to handle async logic.
- Automatically generates 3 actions →
— pending, fulfilled, rejected.



3. createAsyncThunk() - Syntax

```
const asyncThunk = createAsyncThunk(  
  "sliceName/thunkName", // Action type prefix  
  async (arg, thunkAPI) => { // Async logic  
    return data;  
  }  
);
```

arg → any value
you pass while
dispatching.
thunkAPI →
has helpers
like rejectWithValue

4. Example - Fetch Users from API

```
// usersThunk.js
export const fetchUsers = createAsyncThunk(
  'users/fetchUsers',
  async (... , thunkAPI) => {
    try {
      const res = await fetch('https://jsonplaceholder.typicode.com/users');
      if (!res.ok) throw new Error('Failed to fetch');
      const data = await res.json();
      return data;
    } catch (err) {
      return thunkAPI.rejectWithValue(err.message);
    }
  }
);
```

We return
data on success.
On error →
rejectWithValue()
with message.

5. Handle in Slice - extraReducers

```
const usersSlice = createSlice({
  name: 'users',
  initialState: {
    users: [],
    loading: false,
    error: null
  },
  reducers: {},
  extraReducers: (builder) => {
    builder
      .addCase(fetchUsers.pending, (state) => {
        state.loading = true;
        state.error = null;
      })
      .addCase(fetchUsers.fulfilled, (state, action) => {
        state.loading = false;
        state.users = action.payload;
      })
      .addCase(fetchUsers.rejected, (state, action) => {
        state.loading = false;
        state.error = action.payload;
      });
  }
});
```

3 Auto Actions

1. users/fetchUsers/pending
2. users/fetchUsers/fulfilled
3. users/fetchUsers/rejected

State Shape for Async

loading → boolean

users → []

error → string | null

6. Use in Component

```
// ExampleComponent.jsx
const MyComponent = () => {
  const dispatch = useDispatch();
  const { users, loading, error } = useSelector(
    (state) => state.users
  );

  useEffect(() => {
    dispatch(fetchUsers());
  }, [dispatch]);

  if (loading) return <p>Loading....</p>;
  if (error) return <p style={{color: 'red'}}>{error}</p>;

  return (
    <ul>
      {users.map(user => (
        <li key={user.id}>{user.name}</li>
      ))}
    </ul>
  );
};
```

dispatch
the thunk
like a normal
action!

7. Dispatching with Parameters

```
// Pass parameter while dispatching
dispatch(fetchUserById(5)); // 5 → will be available in arg
dispatch(fetchUsers({ page: 1 })); // object also ok
```

8. Error Handling Best Practice

- Always use `rejectWithValue()` to send meaningful error.
- It will be available in `action.payload` in rejected case.

```
return thunkAPI.rejectWithValue("Something went wrong!");
```

9. rejectWithValue() vs throw new Error()

<u>throw new Error()</u>	<u>rejectWithValue()</u>
Sends error in <code>action.error</code> (less control)	Sends custom message in <code>action.payload</code> .
<code>action.error.message</code> will have the message.	You have full control over what error is stored.
Not recommended for expected errors (API errors).	Best practice for API error handling.

10. Other Useful ThunkAPI Helpers

- `thunkAPI.dispatch` → dispatch other actions
- `thunkAPI.getState()` → access current state
- `thunkAPI.rejectWithValue(value)` → handle errors
- `thunkAPI.fulfillWithValue(value, meta?)` → return custom fulfilled value

★ Async Flow Recap ★

`dispatch(thunk)` → pending → API Call → fulfilled (success)
 ↓
 rejected (error)

11. Complete Example in One Place

```
// usersSlice.js (Full Example)
import { createSlice, createAsyncThunk } from "@reduxjs/toolkit";

export const fetchUsers = createAsyncThunk(
  'users/fetchUsers',
  async (_, thunkAPI) => {
    try {
      const res = await fetch('https://jsonplaceholder.typicode.com/users');
      if (!res.ok) throw new Error('Failed to fetch');
      return await res.json();
    } catch (err) {
      return thunkAPI.rejectWithValue(err.message);
    }
  }
);

const usersSlice = createSlice({
  name: 'users',
  initialState: { users: [], loading: false, error: null },
  reducers: {},
  extraReducers: (builder) => {
    builder
      .addCase(fetchUsers.pending, (state) => {
        state.loading = true; state.error = null;
      })
      .addCase(fetchUsers.fulfilled, (state, action) => {
        state.loading = false; state.users = action.payload;
      })
      .addCase(fetchUsers.rejected, (state, action) => {
        state.loading = false; state.error = action.payload;
      });
  }
});

export default usersSlice.reducer;
```

Key Points

- ★ createAsyncThunk handles the async logic.
- ★ It creates 3 actions automatically.
- ★ Handle them in extraReducers.
- ★ Always manage loading & error in state.

Practice more
with different
APIs!



★ QUICK REVISION ★

1. Use `createAsyncThunk` for async logic.
2. It creates 3 actions → pending, fulfilled, rejected.
3. Handle in `extraReducers` using `builder`.
4. Manage loading, success data & error in state.
5. Use `rejectWithValue()` for meaningful errors.
6. Dispatch thunk like a normal action.
7. Access data in component using `useSelector()`.
8. Keep state shape consistent.

STATE SHAPE TEMPLATE

```
{  
  data: [],  
  loading: false,  
  error: null  
}
```

Remember
this
always!



1. What is RTK Query?

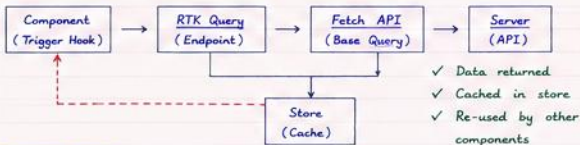
- RTK Query is the data fetching and caching tool built inside Redux Toolkit.
- It simplifies API calls, caching, loading states, error handling, polling, pagination etc.
- Eliminates the need for extra libraries like axios + react-query in most cases.
- Built on top of Redux Toolkit + Immer.

Less code
Auto caching
Loading & error handling built-in
Great DX 🍌

2. Core Concepts

- **Endpoint** → A description of how to fetch (or mutate) data.
- **Query** → For fetching data (GET).
- **Mutation** → For changing data (POST, PUT, DELETE).
- **Cache** → Stores the result of queries in Redux store.
- **Tag** → Used for cache invalidation & refetching.
- **Re-usable Hooks** → Auto generated hooks for each endpoint.

3. How RTK Query Works (High Level Flow)



4. Basic Setup

Step 1: Install (if needed)

```
npm i @reduxjs/toolkit react-redux
```

Step 2: Create an API Slice (api/postsApi.js)

```
import { createApi, fetchBaseQuery } from '@reduxjs/toolkit/query/react'

export const postsApi = createApi({
  reducerPath: 'postsApi',
  baseQuery: fetchBaseQuery({ baseUrl: 'https://jsonplaceholder.typicode.com' }),
  endpoints: (builder) => ({
    getPosts: builder.query({
      query: () => '/posts',
    }),
  }),
})
```

createApi creates a slice with reducer + middleware automatically

Step 3: Add to Store (store.js)

```
import { configureStore } from '@reduxjs/toolkit'
import { postsApi } from './api/postsApi'

export const store = configureStore({
  reducer: {
    [postsApi.reducerPath]: postsApi.reducer,
  },
  middleware: (getDefaultMiddleware) =>
    getDefaultMiddleware().concat(postsApi.middleware),
})
```

Don't forget to add middleware!

Step 4: Provide Store

```
<Provider store={store}>
  <App />
</Provider>
```

Now RTK Query is ready to use in the app 🍌

5. Endpoints - Query vs Mutation

Query (GET)	Mutation (POST/PUT/DELETE)
- Read / fetch data - Cached automatically - Auto refetch on certain events - Uses <code>builder.query()</code>	- Change / update data - Does not cache result - You usually invalidate tags - Uses <code>builder.mutation()</code>

6. Multiple Endpoints Example

```
export const postsApi = createApi({
  reducerPath: 'postsApi',
  baseQuery: fetchBaseQuery({ baseUrl: 'https://jsonplaceholder.typicode.com' }),
  tagTypes: ['Post'],
  endpoints: (builder) => ({
    getPosts: builder.query({
      query: () => '/posts',
      providesTags: (result) =>
        result
          ? [
              ...result.map(({ id }) => ({ type: 'Post', id })),
              { type: 'Post', id: 'LIST' },
            ]
          : [{ type: 'Post', id: 'LIST' }],
    }),
    addPost: builder.mutation({
      query: (newPost) => ({
        url: '/posts',
        method: 'POST',
        body: newPost,
      }),
      invalidatesTags: [{ type: 'Post', id: 'LIST' }],
    }),
  }),
})
```

providesTags →
Tells what data
this query provides

invalidatesTags →
Marks cache as
stale & triggers
refetch.

7. Auto Generated Hooks

Endpoint Type	Example Endpoint	Auto Generated Hook
Query	<code>getPosts</code>	<code>useGetPostsQuery()</code>
Mutation	<code>addPost</code>	<code>useAddPostMutation()</code>

- Hooks are generated based on endpoint name.
- Use them directly in components.

8. Using in Component

Example - Fetching Data

```
import { useGetPostsQuery } from './api/postsApi'

function Posts() {
  const { data, isLoading, isError, error } = useGetPostsQuery()
  if (isLoading) return <p>Loading...</p>
  if (isError) return <p>Error: {error.toString()}</p>
  return (
    <ul>
      {data?.map(post) => (
        <li key={post.id}>{post.title}</li>
      )}
    </ul>
  )
}
```

data → result

isLoading →
loading state

isError → error
state

error → error
object

9. Using Mutation in Component

```
import { useAddPostMutation } from './api/postsApi'

function AddPost() {
  const [addPost, { isLoading, isSuccess, isError }] = useAddPostMutation()

  const handleAdd = () => {
    addPost({
      title: 'foo',
      body: 'bar',
      userId: 1,
    })
  }

  return (
    <div>
      <button onClick={handleAdd} disabled={isLoading}
        {isLoading ? 'Adding...' : 'Add Post'}
      </button>
      {isSuccess && <p>✅ Post Added!</p>}
      {isError && <p>❌ Failed to add post</p>}
    </div>
  )
}
```

Mutation returns a trigger function + status flags.

10. Cache & Invalidation

- Queries are cached based on endpoint + parameters.
- By default cache is kept as long as component is mounted.
- Use `invalidatesTags` in mutation to refetch related queries.
- You can also use `refetch()` manually.

11. Useful Options in Endpoints

Option	Purpose
<code>query</code>	Defines the API endpoint.
<code>providesTags</code>	Defines what data this query provides.
<code>invalidatesTags</code>	Invalidates related cache.
<code>transformResponse</code>	Modifies the response data.
<code>keepUnusedDataFor</code>	Time (in sec) to keep data in cache after component unmount.

12. Benefits

- ✓ Less boilerplate code
- ✓ Built-in caching
- ✓ Auto refetching
- ✓ Loading & error handling
- ✓ Pagination, polling, refetch on focus
- ✓ Tag based cache invalidation
- ✓ Great Developer Experience.

13. Quick Revision

1. `createApi()` → Create API slice.
2. Define endpoints with `builder.query / builder.mutation`.
3. Add reducer + middleware to store.
4. Use auto generated hooks in components.
5. Enjoy caching, refetching & less boilerplate! 🎉



ZUSTAND - COMPLETE NOTES

Page 1

1. What is Zustand ?

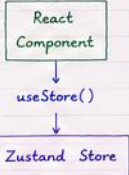
- Zustand is a small, fast and scalable state management library for React.
- It uses a simple store based approach.
- No Providers, No Complex Setup.
- Combines simplicity of useState with power of Redux.

Small
Fast
Simple



2. Core Concepts

- **Store** → Central place to hold state.
- **useStore()** → Hook to access store.
- **get()** → Read state outside components.
- **set()** → Update state.
- **Action** → Functions that update state.



3. Basic Store Example

```
import { create } from 'zustand'
// create a store
const useCounterStore = create((set) => ({
  count: 0,
  increment: () => set((state) => ({ count: state.count + 1 })),
  decrement: () => set((state) => ({ count: state.count - 1 })),
  reset: () => set({ count: 0 })
}))
```

Note:

- **set()** merges state by default.
- You can also return an object to update.

Explanation

- **count** → initial state
- **increment()** → increases count
- **decrement()** → decreases count
- **reset()** → sets count to 0

Looks like useState,
Works like Redux !

4. Using Store in Components

```
import React from 'react'
import useCounterStore from './store/counterStore'

function Counter() {
  const count = useCounterStore((state) => state.count)
  const { increment, decrement, reset } = useCounterStore()
  return (
    <div>
      <h2>Count: {count}</h2>
      <button onClick={increment}>+1</button>
      <button onClick={decrement}>-1</button>
      <button onClick={reset}>Reset</button>
    </div>
  )
}
```

useStore()
subscribes only
to the used
state (Optimized
re-renders).

5. Advanced: Using get()

// get() helps to read current state inside actions

```
const useUserStore = create((set, get) => ({
  name: 'Kapil',
  updateName: (newName) => set({ name: newName }),
  capitalizeName: () => {
    const current = get().name
    set({ name: current.toUpperCase() })
  }
}))
```

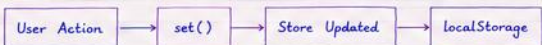
get()
reads current
state inside
actions.

6. Persist Middleware (State Persistence)

```
import { create } from 'zustand'
import { persist } from 'zustand/middleware'

const usePersistStore = create(
  persist(
    (set) => ({
      count: 0,
      increment: () => set((state) => ({ count: state.count + 1 })),
    }),
    { name: 'my-store' } // store name in localStorage
  )
)
```

State will
be saved in
localStorage
automatically.



(persist saves)

7. Devtools Middleware (Redux DevTools Support)

```
import { devtools } from 'zustand/middleware'

const useDevtoolsStore = create(
  devtools((set) => ({
    count: 0,
    increment: () => set((state) => ({ count: state.count + 1 })),
  })),
)
```

Check state
changes in
Redux
DevTools.

8. Combining Middlewares

```
import { create } from 'zustand'
import { persist } from 'zustand/middleware'
import { devtools } from 'zustand/middleware'

const useStore = create(
  devtools(
    persist(
      (set) => ({
        count: 0,
        increment: () => set((state) => ({ count: state.count + 1 })),
      })),
      { name: 'my-store' }
    )
  )
)
```

Order
devtools(
 persist(
 yourStore
)
)

Outer → devtools
Inner → persist

9. Immutable Updates

- Zustand allows mutable style updates.
- But always return new object (best practice).

Always
return new
object / array.

```
const useTodoStore = create((set) => ({
  todos: [],
  addTodo: (text) => set((state) => ({
    todos: [...state.todos, { id: Date.now(), text: text, done: false }]
  })),
  toggleTodo: (id) => set((state) => ({
    todos: state.todos.map(t) =>
      t.id === id ? { ...t, done: !t.done } : t
  })),
}))
```

10. Subscribing Outside Components

```
// Useful for non-React logic
const unsub = useCounterStore.subscribe((state) =>
  console.log('Count changed.', state.count)
)

// later call to stop
unsub()
```

subscribe()
to listen
state changes
outside UI.

11. Advantages of Zustand

- ✓ Very small size (~1KB)
- ✓ Simple API
- ✓ No Providers
- ✓ Fast performance
- ✓ Works well with TypeScript
- ✓ DevTools support
- ✓ Middlewares for advanced needs

12. When to Use Zustand ?

- Small to medium applications.
- When you want simple and lightweight state management.
- When Redux feels heavy.
- When you want good performance + less boilerplate.

QUICK REVISION

Read State

```
const value = useStore(
  (state) => state.value
)
```

Outside Component

```
const value = useStore.getState()
```

Subscribe

```
const unsub = useStore.subscribe(
  (state) => console.log(state)
)

unsub() // stop
```

Update State

```
set((state) => ({
  value: newValue
})))
```

Flow

Action



set()



Store Update



Subscriber
(Component)
re-renders

★ Zustand = Simplicity + Performance + Power ★

REDUX TOOLKIT (RTK) vs ZUSTAND

Page 1

BASICS COMPARISON

★ 1. What are they?

- RTK → Official, opinionated, batteries-included state management for large-scale apps.
- Zustand → Small, simple, unopinionated store for React. Uses hook-based API.

Both
Manage
State!



★ 2. Core Philosophy

<u>Redux Toolkit (RTK)</u>	<u>Zustand</u>
Follow structure and patterns Best for predictability and maintainability.	Minimal API Do what you need, no extra rules.

★ 3. Boilerplate Code

RTK → More setup

- Store
- Slice
- Actions
- Provider

Zustand → Very less setup

- Create store
- Use store in component

★ 4. Setup Example

RTK Setup Flow

```
createSlice()
  ↓
configureStore()
  ↓
<Provider store={store}>
  ↓
useSelector / useDispatch()
```

Zustand Setup Flow

```
create()
  ↓
useStore()
  ↓
Done ✓
```

★ 5. State Access in Components

RTK

```
const value = useSelector(
  (state) => state.slice.value
)
```

```
const dispatch = useDispatch()
```

Zustand

```
const value = useStore(
  (state) => state.value
)
```

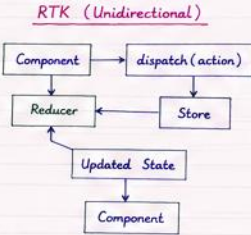
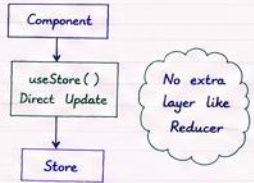
Direct hook usage!
No dispatch needed
for simple updates.



★ 6. State Update

<u>RTK</u>	<u>Zustand</u>
<code>dispatch(increment())</code> <code>dispatch(setValue(10))</code>	<code>const increment =</code> <code>useStore((state) → state.increment)</code> <code>increment()</code>
Updates are done by dispatching actions.	Call function directly from the store.

★ 7. Data Flow

Zustand

★ 8. Best For

<u>RTK</u>	<u>Zustand</u>
Large applications	Small to medium applications
Complex state logic	Simple state logic
Teams who like structure	Teams who like simplicity and flexibility
Need devtools, middleware, RTK Query, etc.	Need lightweight state management

★ 9. Middleware & DevTools

<u>RTK</u>	<u>Zustand</u>
✓ Built-in support	✓ Devtools middleware
✓ Rtk Toolkit DevTools	✓ Persist middleware
✓ Middleware ecosystem	✓ Very flexible

★ 10. Learning Curve

<u>RTK</u>
Medium

Zustand is easier for beginners.

<u>Zustand</u>
Easy

★ 11. Performance

Both are fast ⚡ but Zustand is slightly lighter (smaller bundle size).



★ 12. Feature Comparison

<u>Feature</u>	<u>RTK</u>	<u>Zustand</u>
State Management	Yes	Yes
Boilerplate	More	Very Less
Setup	Complex	Simple
DevTools	Built-in	Middleware
Middleware	Built-in support	Manual add
Async Logic (RTK Query)	Yes	No (Manual)
Persist State	Yes	Yes (Middleware)
Best For	Large & Complex Apps	Small & Medium Apps
Learning Curve	Medium	Easy
Flexibility	Structured	Highly Flexible

★ 13. When to use What?

Use RTK when:

- Building large scale app.
- Need strict structure.
- Need RTK Query for API.
- Working in a big team.

VS

Use Zustand when:

- Building small / medium app.
- Need simple global state.
- Want less boilerplate.
- Like flexible approach.

★ 14. Quick Notes

- ✓ RTK → Structure + Convention + Powerful.
Good for long-term large projects.
- ✓ Zustand → Simplicity + Flexibility + Lightweight.
Good for quick and modern apps.

☑ Both are great!
Choose based on project size and team preference.



★ 15. One Line Summary

RTK

More boilerplate,
More power,
Better for big apps.

VS

Zustand

Less code,
More flexibility,
Better for simple apps.

★ RIGHT TOOL, RIGHT TIME, RIGHT PROJECT ★